

A User-Centred, From-Scratch Scientific Calculator for the Tangent Function $\tan(x)$

Deliverable 2: From-Scratch Implementation, Tkinter GUI, Version Control, Updated
Requirements

Risat Ahmed
Student ID: 40294116
SOEN 6011 – Software Engineering Processes, Section CC
Concordia University – Summer 2026

July 29, 2026

Contents

1	Introduction	3
2	Problem 5 — From-Scratch Core and Tkinter GUI	3
2.1	Interpreting the from-scratch boundary	3
2.2	From-scratch π	3
2.3	Numerical architecture	4
2.4	Tkinter GUI	4
3	Problem 6 — Repository, Commit History, README	5
3.1	Repository structure	5
3.2	Commit history	6
3.3	README	6
4	Problem 7 — Updated Requirements (Baseline v0.2)	6
5	Verification	6
6	Decision Rationale	8
7	Implementation Summary	8
A	GAI Prompt Log (summary)	8

1 Introduction

This report documents Deliverable 2 (D2) of the tangent-function calculator project, which modifies the D1 implementation and updates the D1 requirements per the project’s dependency chain (*Persona* → *Requirements* → *Two Algorithms* → *Algorithm Selection* → *CLI Prototype* → *From-Scratch GUI* → *Updated Requirements*). D1 delivered a persona, an ISO/IEC/IEEE 29148 requirements baseline (v0.1), two independent algorithms in pseudocode, and a Python CLI prototype that already computed sin/cos itself but still imported `PI = math.pi`. D2 covers three problems: replacing every remaining prohibited mathematical dependency with a from-scratch implementation and adding a Tkinter GUI (Problem 5); publishing a public version-controlled repository with a curated commit history and README (Problem 6); and revising the requirements baseline to v0.2 using evidence from the D2 implementation (Problem 7).

Per constraint C-03, every problem below used a public GAI tool (Claude Code, Sonnet 5) with a documented prompt, output evaluation, and verification method; the full log is Appendix A.

Public repository: <https://github.com/risatahmed/soen6011-tan-calculator> (verified reachable from a logged-out session; Section 3).

2 Problem 5 — From-Scratch Core and Tkinter GUI

2.1 Interpreting the from-scratch boundary

The D1 CLI already avoided `math.sin/math.cos` (both were computed via a Maclaurin series), leaving exactly one prohibited dependency: `PI = math.pi`. D2 first froze an explicit interpretation (`docs/from_scratch_boundary.md`, decision D-06): **prohibited** is a library call that itself computes a trigonometric/transcendental result (`math.sin/cos/tan/atan/pi`); **permitted** is ordinary arithmetic (`+` `-` `*` `/`, `abs()`, `round()`), I/O, and the Tkinter toolkit. Under this interpretation, Algorithm A’s Maclaurin series — which never calls a trig library function, only arithmetic — already satisfies the from-scratch constraint once `PI` itself is derived independently. D2 therefore **retains Algorithm A** rather than switching to Algorithm B (CORDIC), reversing the D1 report’s forward-looking note; switching this late would add a new $\arctan(2^{-i})$ table and ~ 40 -iteration loop without being required for compliance. A `grep` pass over `src/` (Table 1) confirms no `math.*` trig/pi symbol remains.

Command	Result
<code>grep -rn "math\." src/</code>	No matches outside docstring prose
<code>grep -rn "^import math" src/</code>	No matches

Table 1: From-scratch boundary verification (full inventory in `docs/from_scratch_boundary.md`).

2.2 From-scratch π

`src/constants.py` derives `PI` from Machin’s 1706 identity, using only a hand-written arctan Maclaurin series (decision D-07):

$$\frac{\pi}{4} = 4 \arctan\left(\frac{1}{5}\right) - \arctan\left(\frac{1}{239}\right), \quad \arctan(y) = y - \frac{y^3}{3} + \frac{y^5}{5} - \dots$$

Both arguments ($1/5$, $1/239$) are $\ll 1$, so the series converges in ~ 13 terms to 10^{-18} accuracy. The resulting value agrees with `math.pi` to 8.9×10^{-16} (measured directly, Section 5), which is below double-precision unit roundoff for values of this magnitude.

2.3 Numerical architecture

The D1 monolithic `cli.py` was split into single-responsibility modules, per D2-P5.2:

- `constants.py` — from-scratch PI.
- `exceptions.py` — `NumericalRangeError` (VAL-03), `DomainError` (VAL-04), `ConvergenceError` (new: a series that exhausts `max_terms` without reaching tolerance now raises instead of silently returning an unconverged value, closing a REL-01 gap in D1).
- `trig_series.py` — `maclaurin_sin/maclaurin_cos`, unchanged recurrence from D1, now raising `ConvergenceError` on exhaustion.
- `validation.py` — input parsing (`parse_x`) and unit conversion (`to_radians`, degrees via decision D-08).
- `tan_core.py` — the pure entry point `tan(x, tolerance, max_terms)`: range-reduces modulo $\pi/2$ using the from-scratch PI, evaluates the reduced angle's sin/cos, recombines per quadrant, and guards the division against a near-zero denominator.

Listing 1: Pure core entry point, `src/tan_core.py` (excerpt)

```

1 def tan(x, tolerance=SERIES_TOL, max_terms=SERIES_N_MAX,
2       x_max=X_MAX, pole_epsilon=POLE_EPSILON):
3     if abs(x) > x_max:
4         raise NumericalRangeError(...)
5     sign = 1.0
6     reduced_x = x
7     if reduced_x < 0:
8         reduced_x = -reduced_x
9         sign = -1.0 # tan(-x) = -tan(x)
10    k = round(reduced_x / (PI / 2))
11    r = reduced_x - k * (PI / 2) # r in [-pi/4, pi/4]
12    quadrant_flip = (k % 2) == 1
13    sin_r = maclaurin_sin(r, tolerance, max_terms)
14    cos_r = maclaurin_cos(r, tolerance, max_terms)
15    numerator, denominator = (-cos_r, sin_r) if quadrant_flip else (sin_r, cos_r)
16    if abs(denominator) < pole_epsilon:
17        raise DomainError(...)
18    return sign * (numerator / denominator)

```

`tan_core.py` has no I/O and no GUI dependency, so it is directly importable and testable in isolation (D2-P5.2's "testable without launching GUI"); both `cli.py` and `gui.py` import it as their only source of numerical computation.

2.4 Tkinter GUI

`docs/gui_wireframe.md` was written before `gui.py`: an x entry with a unit-aware label ("x in radians"/"x in degrees"), a radians/degrees radio-button pair (radians default), a static domain-hint label shown *before* any error can occur, Calculate/Clear buttons bound to **Enter**/**Esc** at the window level, and a result/status pair where every status message is prefixed with the literal word "Error" or "OK" — never colour alone. Figure 1 shows five captured states. The Calculate handler is the

only code in `gui.py` that touches `src.tan_core/src.validation`; it catches `NumericalRangeError`, `DomainError`, `ConvergenceError`, and `ValueError` individually and turns each into a specific status message, so no exception ever reaches the user as a raw traceback (D2-P5.5).

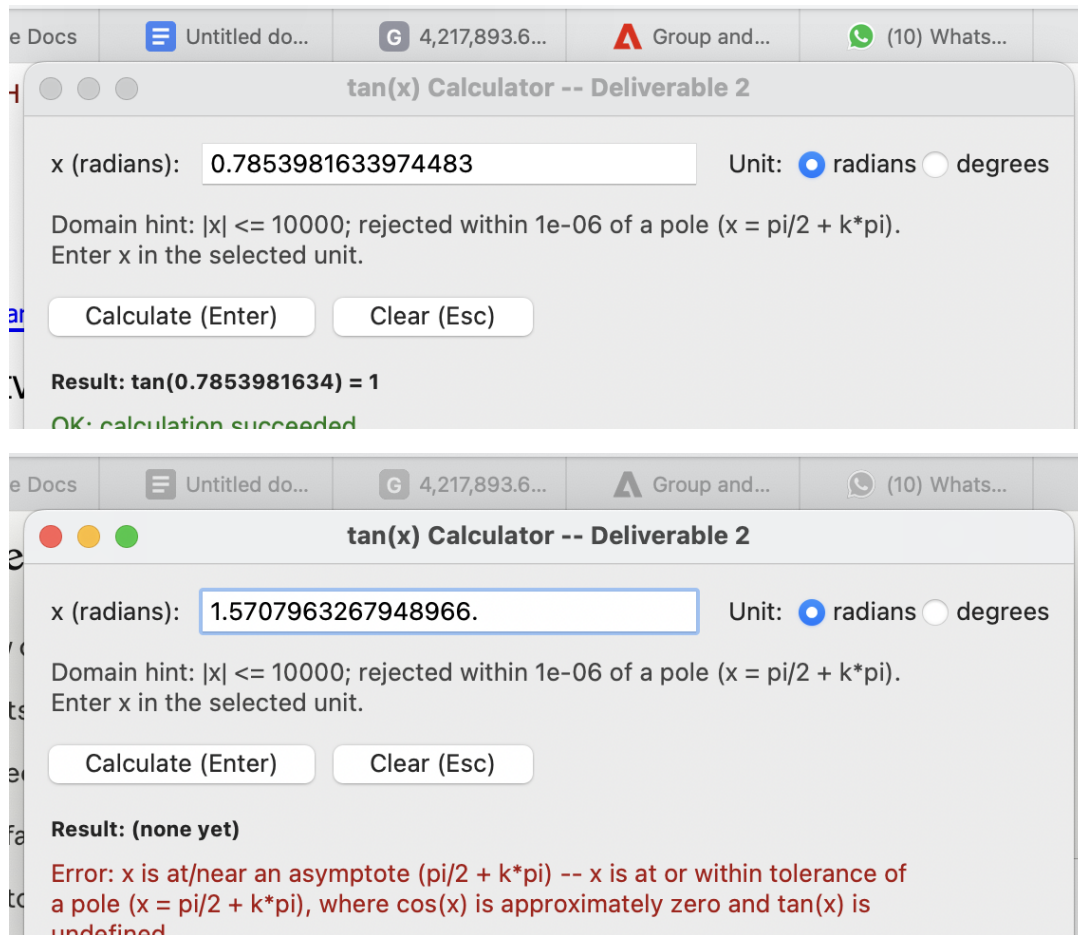


Figure 1: GUI states: valid result for $x = \pi/4$ (top); near-pole rejection at $x = \pi/2$ with a specific, non-colour-only error message (bottom). Three further states (degrees mode, out-of-range, invalid text) are in `docs/screenshots/`.

3 Problem 6 — Repository, Commit History, README

3.1 Repository structure

Public repository: <https://github.com/risatahmed/soen6011-tan-calculator> (created and pushed via `gh repo create/git push`; reachability confirmed with an unauthenticated `curl` request returning HTTP 200, equivalent to a logged-out browser visit).

The repository root now contains `README.md`, `src/` (7 modules), `tests/` (the D2 manual verification script; the graded PyUnit suite is D3), and `docs/` (persona, requirements v0.1 *and* v0.2, algorithms, decision log, GAI log, the new from-scratch boundary and GUI wireframe docs, and screenshots), matching the task list's suggested layout.

3.2 Commit history

D1 was submitted as a Moodle zip before this Git repository was initialized, so it has no incremental history of its own; that limitation is stated honestly in the first commit message rather than hidden or fabricated with backdated commits. From that point forward, D2 work is committed as eleven cohesive, per-concern commits (Table 2) rather than one bulk upload, each with an imperative subject and a body explaining motivation/trade-offs where relevant (D2-P6.2).

Commit subject (chronological)
Add Deliverable 1 baseline: persona, requirements v0.1, algorithms, CLI
D2-P5.1: document the from-scratch dependency boundary
D2-P5.2/5.3: implement the from-scratch numerical core
D2: migrate cli.py onto the shared from-scratch core
D2-P5.4: add the Tkinter GUI wireframe
D2-P5.5: implement the Tkinter GUI
D2-P5.6: add verification suite, capture CLI/GUI evidence
D2-P6.3: add project README
D2-P7.1: update requirements to baseline v0.2
Log D2 decisions and GAI prompts
D2-REP: add D2 LaTeX report and slides

Table 2: Commit history from repository initialization through this report (`git log -oneline`).

3.3 README

README.md states the project’s purpose, the from-scratch scope, exact run commands (`python3 -m src.cli`, `python3 -m src.gui` — module invocation, since the package uses absolute `src.*` imports), usage examples with screenshot references, an error-message/troubleshooting table, the repository structure, and known limitations stated up front (the 10^4 range bound, the near-pole accuracy margin, and that the formal unit-test suite is a D3 deliverable) rather than only discovered by a reader later.

4 Problem 7 — Updated Requirements (Baseline v0.2)

Every v0.1 requirement was compared against the D2 implementation and `docs/verification_matrix_d2.md`. Table 3 summarizes the changes; 13 v0.1 requirements (FR-01/03/04/05/06, VAL-01–04, ACC-02, USE-01, REL-01/02, ERR-01/02, DOC-01) carry forward **unchanged** and remain in force.

No requirement was deleted; the v0.1 file is retained on disk unmodified, per constraint C-06 (iterative consistency). ACC-01 was deliberately *not* weakened to make the near-pole margin more comfortable — it was documented instead, following the same "don’t weaken tolerances without justification" principle the task list applies to D3 unit testing.

5 Verification

`tests/manual_verification_d2.py` runs 15 cases against `math.tan` computed independently (never the from-scratch algorithm verifying itself) — special angles, odd symmetry, period- π reduction, both near-pole directions, both out-of-range directions, and invalid/empty input. All 15 pass within the ACC-01 10^{-9} relative-error bound (Table 4); this is a standalone verification script, not the formal PyUnit suite (D3-P8.2).

ID	Change	What changed	Discovered via
FR-02	Revised	Angle unit stated <i>and</i> selectable (radians default).	Implementation (D-08)
FR-07	New	Labelled degrees input mode.	Implementation (D-08)
USE-02	Revised	"Textual interface" broadened to "at least one interface."	GUI added (D2-P5.5)
USE-03	New	Tkinter GUI with labelled controls, keyboard nav, no colour-only errors.	D2-P5.4/5.5
DEP-01	New	Core shall not call <code>math.sin/cos/tan/pi</code> .	D2-P5.1/5.2/5.3
ERR-03	New	Raise one of 3 named exception types, not a bare <code>Exception</code> .	D2 architecture note
ACC-01	Revised (evidence)	Bound unchanged; flagged as tight near the pole boundary (3.83×10^{-10} measured vs. 10^{-9} ceiling).	D2-P5.6 verification
REL-03	New	No traceback across 5 documented failure modes (adds convergence).	D2-P5.5/5.6
DOC-02	New	Document the from-scratch boundary and keep it current.	D2-P5.1

Table 3: v0.1 $\rightarrow$ v0.2 changelog (full table in docs/requirements/requirements_v0.2.md).

Table 4: Verification highlights (full 15-case matrix in docs/verification_matrix_d2.md).

Case	Result	Rel. error	Verdict
$x = \pi/4$	1.0000000000000013	1.44×10^{-15}	Pass
$x = -\pi/4$ (odd symmetry)	-1.0000000000000013	1.44×10^{-15}	Pass
$x = \pi + \pi/4$ (period π)	0.9999999999999996	1.11×10^{-16}	Pass
$x = 1000$	1.4703241557018916	5.63×10^{-13}	Pass
$x = \pi/2 - 10^{-6}$ (near pole)	999999.999637844	3.83×10^{-10}	Pass (tightest margin)
$x = \pi/2$ (at pole)	raised <code>DomainError</code>	n/a	Pass (rejected)
$x = 10^{10}$	raised <code>NumericalRangeError</code>	n/a	Pass (rejected)
"abc" / ""	raised <code>ValueError</code>	n/a	Pass (rejected)

Known limitations (honest disclosure): the $|x| \leq 10^4$ range bound is unchanged and still provisional pending professor confirmation; the near-pole relative error (3.83×10^{-10}) is the closest margin of any passing case to the 10^{-9} ceiling, confirming D1's prediction that precision degrades near the guard-band boundary; no performance/timing requirement is verified (deliberately out of scope).

6 Decision Rationale

Key D2 decisions from `docs/decisions/decision_log.md`:

- **D-06:** retain Algorithm A rather than switch to CORDIC — ordinary arithmetic is not prohibited "mathematical support," only library trig calls are, so Algorithm A already satisfies the from-scratch constraint once PI is independently derived.
- **D-07:** derive PI via Machin's arctangent identity rather than a literal digit string, reusing the existing tolerance/max-terms series pattern.
- **D-08:** add an optional, clearly-labelled degrees mode, since the task list explicitly permits it "if time permits" and it directly serves the persona's stated radians/degrees confusion.

7 Implementation Summary

D2 delivers a fully from-scratch numerical core (Table 1), a Tkinter GUI sharing that core with the CLI, a public version-controlled repository with an eleven-commit incremental history, a README a stranger can run the project from, and a revised requirements baseline (v0.2) with a traceable changelog. All figures, tables, and verification results in this report were produced by the exact source committed as D2-REP in the repository history.

A GAI Prompt Log (summary)

Per constraint C-03, the full CASTROFF-informed prompts and evaluation log for Problems 5–7 are in `docs/prompts/gai_prompt_log.md` (D2 section). All three problems used the same tool: Claude Code (Claude, Anthropic, Sonnet 5 model), accessed 2026-07-29. In every case at least one part of the raw output was corrected before being adopted: Problem 5's decision to keep Algorithm A rather than follow D1's own forward-looking note to switch to CORDIC (re-reading the actual from-scratch restriction); the Maclaurin series functions were changed from D1's silent-stop behaviour to raising `ConvergenceError`, since the D2 architecture note names that exception explicitly; and an early GUI draft that put exception-message formatting inside `tan_core.py` was moved out, keeping the numerical core GUI-independent.

References

- [1] A. Cooper, R. Reimann, and D. Cronin, *About Face 3: The Essentials of Interaction Design*. Wiley, 2007.
- [2] J. Pruitt and T. Adlin, *The Persona Lifecycle: Keeping People in Mind Throughout Product Design*. Morgan Kaufmann, 2006.
- [3] ISO/IEC/IEEE, *ISO/IEC/IEEE 29148:2018 – Systems and Software Engineering – Life Cycle Processes – Requirements Engineering*, 2018.
- [4] J. E. Volder, "The CORDIC Trigonometric Computing Technique," *IRE Transactions on Electronic Computers*, vol. EC-8, no. 3, pp. 330–334, 1959.
- [5] J.-M. Muller et al., *Handbook of Floating-Point Arithmetic*, 2nd ed. Birkhäuser, 2016.
- [6] J. Machin's arctangent identity for π , as presented in standard numerical-analysis references; see also NIST DLMP below.

-
- [7] M. Abramowitz and I. A. Stegun, *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. National Bureau of Standards, 1964.
- [8] NIST, *NIST Digital Library of Mathematical Functions*, <https://dlmf.nist.gov/>, 2026.